

enhanced Portfolio Risk Assessment Using Value at Risk (VaR)

February 12, 2025

```
[3]: import yfinance as yf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm
import plotly.express as px
```

```
[5]: # Define a larger and diversified portfolio
stocks = [
    'AAPL', 'MSFT', 'GOOGL', 'AMZN', 'TSLA', # Tech
    'JNJ', 'PFE', 'MRK', # Healthcare
    'XOM', 'CVX', 'COP', # Energy
    'WMT', 'TGT', 'COST', # Retail
    'JPM', 'BAC', 'GS', # Financials
    'DIS', 'NFLX', # Entertainment
    'SPY' # S&P 500 ETF
]
weights = np.array([0.05] * 20) # Equal-weighted portfolio
initial_investment = 1000000 # $1,000,000 initial investment
```

```
[7]: # Fetch historical data
data = yf.download(stocks, start="2020-01-01", end="2023-01-01")['Adj Close']

# Calculate daily returns
returns = data.pct_change().dropna()
```

[*****100%*****] 20 of 20 completed

```
[9]: # Calculate portfolio returns
portfolio_returns = (returns * weights).sum(axis=1)
```

```
[11]: # Historical Simulation VaR
confidence_level = 0.95
var_1day = np.percentile(portfolio_returns, (1 - confidence_level) * 100)
var_1day_dollar = initial_investment * abs(var_1day)

print(f"1-day 95% VaR (Historical Simulation): ${var_1day_dollar:.2f}")
```

1-day 95% VaR (Historical Simulation): \$25094.20

```
[13]: # Monte Carlo Simulation for VaR
def monte_carlo_var(returns, weights, initial_investment, confidence_level=0.
    ↪95, simulations=10000):
    mean_returns = returns.mean()
    cov_matrix = returns.cov()
    portfolio_std = np.sqrt(np.dot(weights.T, np.dot(cov_matrix, weights)))

    # Simulate portfolio returns
    simulated_returns = np.random.normal(mean_returns.dot(weights),
    ↪portfolio_std, simulations)

    # Calculate VaR
    var_mc = np.percentile(simulated_returns, (1 - confidence_level) * 100)
    var_mc_dollar = initial_investment * abs(var_mc)

    return var_mc, var_mc_dollar

var_mc, var_mc_dollar = monte_carlo_var(returns, weights, initial_investment)
print(f"1-day 95% VaR (Monte Carlo Simulation): ${var_mc_dollar:.2f}")
```

1-day 95% VaR (Monte Carlo Simulation): \$25456.83

```
[15]: # Variance-Covariance Method for VaR
def var_cov_var(returns, weights, initial_investment, confidence_level=0.95):
    portfolio_returns = returns.dot(weights)
    mean_return = portfolio_returns.mean()
    std_return = portfolio_returns.std()

    # Calculate VaR using the normal distribution
    z_score = norm.ppf(1 - confidence_level)
    var_vcv = mean_return + z_score * std_return
    var_vcv_dollar = initial_investment * abs(var_vcv)

    return var_vcv, var_vcv_dollar

var_vcv, var_vcv_dollar = var_cov_var(returns, weights, initial_investment)
print(f"1-day 95% VaR (Variance-Covariance): ${var_vcv_dollar:.2f}")
```

1-day 95% VaR (Variance-Covariance): \$25698.38

```
[17]: # Stress Testing: Simulate a 20% market crash
stress_factor = 0.20
stress_returns = returns * (1 - stress_factor)

# Calculate portfolio returns under stress
stress_portfolio_returns = stress_returns.dot(weights)
```

```
# Calculate VaR under stress
stress_var = np.percentile(stress_portfolio_returns, (1 - confidence_level) * 100)
stress_var_dollar = initial_investment * abs(stress_var)
print(f"Stress Test 1-day 95% VaR (20% crash): ${stress_var_dollar:.2f}")
```

Stress Test 1-day 95% VaR (20% crash): \$20075.36

```
[19]: # Conditional VaR (CVaR)
cvar = portfolio_returns[portfolio_returns <= var_1day].mean()
cvar_dollar = initial_investment * abs(cvar)
print(f"1-day 95% CVaR: ${cvar_dollar:.2f}")
```

1-day 95% CVaR: \$39576.99

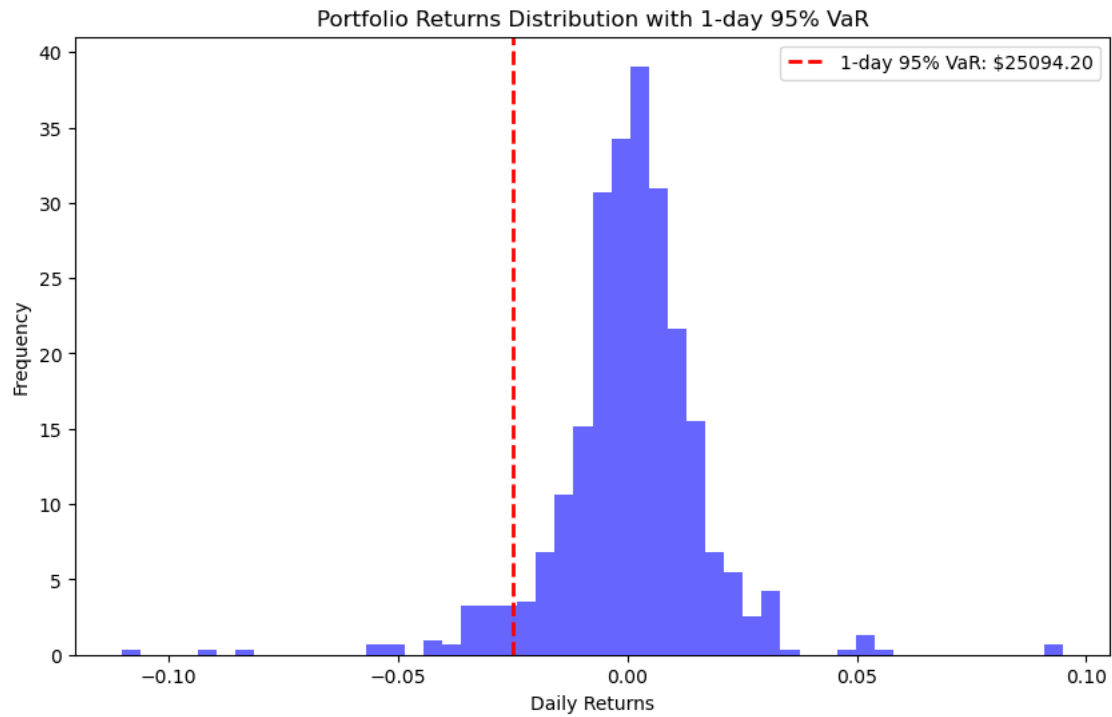
```
[21]: # Sharpe Ratio
risk_free_rate = 0.02 / 252 # Annual risk-free rate (2%) converted to daily
sharpe_ratio = (portfolio_returns.mean() - risk_free_rate) / portfolio_returns.std()
print(f"Sharpe Ratio: {sharpe_ratio:.2f}")
```

Sharpe Ratio: 0.04

```
[23]: # Maximum Drawdown
cumulative_returns = (1 + portfolio_returns).cumprod()
peak = cumulative_returns.expanding(min_periods=1).max()
drawdown = (cumulative_returns - peak) / peak
max_drawdown = drawdown.min()
print(f"Maximum Drawdown: {max_drawdown * 100:.2f}%")
```

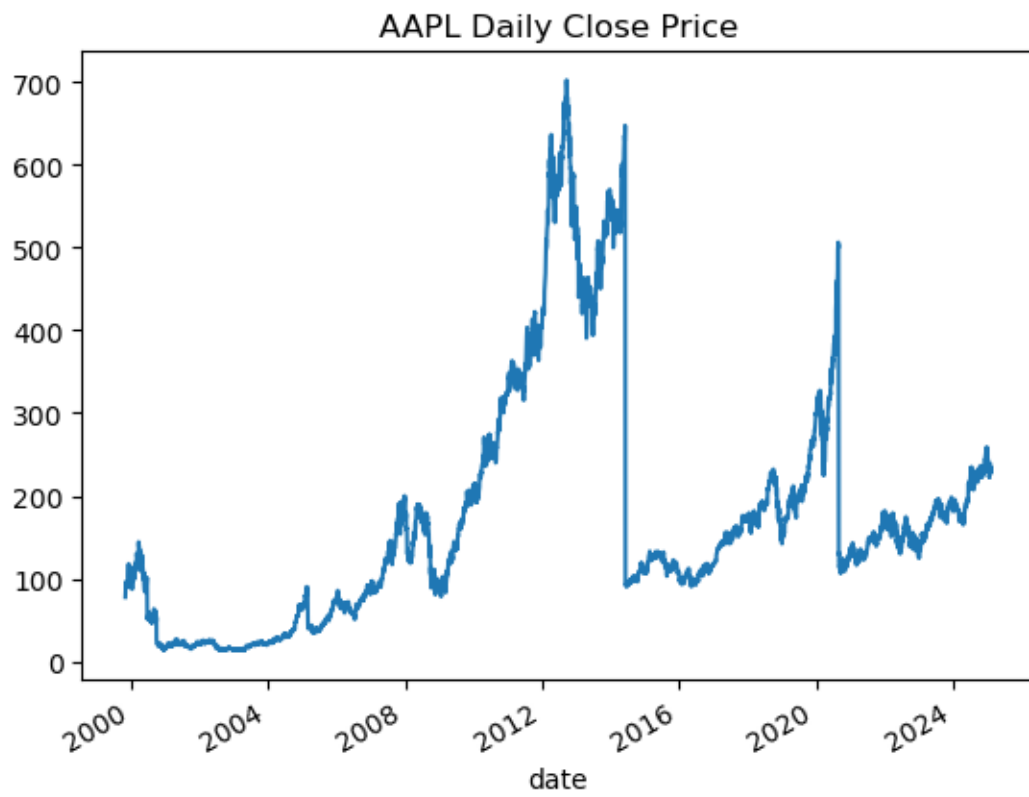
Maximum Drawdown: -32.09%

```
[25]: # Plot histogram of portfolio returns
plt.figure(figsize=(10, 6))
plt.hist(portfolio_returns, bins=50, density=True, alpha=0.6, color='b')
plt.axvline(var_1day, color='r', linestyle='dashed', linewidth=2, label=f'1-day 95% VaR: ${var_1day_dollar:.2f}')
plt.title('Portfolio Returns Distribution with 1-day 95% VaR')
plt.xlabel('Daily Returns')
plt.ylabel('Frequency')
plt.legend()
plt.show()
```



```
[27]: # Fetch real-time data using Alpha Vantage
from alpha_vantage.timeseries import TimeSeries
api_key = 'API' # Replace with your Alpha Vantage API key
ts = TimeSeries(key=api_key, output_format='pandas')

# Fetch data for a single stock (e.g., AAPL)
data, meta_data = ts.get_daily(symbol='AAPL', outputsize='full')
data['4. close'].plot(title='AAPL Daily Close Price')
plt.show()
```



[]: